

משך הבחינה: 180 דקות
מספר שאלות: 6 יש לענות על כולן
חומר עזר: חומר סגור
ניתן להשתמש בדפי טיוטה
אופן הגשת התשובות: הקלדה + סריקה

קראו את כל הבחינה בעיון ורק לאחר מכן תתחילו לענות
יש לכתוב תשובות ברורות וקריאות.
בשאלות הסבר יש לענות בקצרה ולנמק באמצעות המושגים שנלמדו בקורס
יש לכתוב קוד תקין, קריא ומדויק. אין צורך להעתיק מחדש קוד שכבר מופיע בשאלה
אם לא נאמר אחרת, ניתן להניח שכל הספריות הדרושות נכללו ושנעשה שימוש במרחב השמות
התקני: std::
ניתן להוסיף למחלקות פונקציות עזר פרטיות
בשאלות שבהן נדרש פלט, יש לכתוב את הפלט המדויק לפי סדר השורות

בהצלחה רבה!!

1. נתונה המחלקה הבאה והפונקציות המשתמשות בה:

```
#include <iostream>
#include <string>
using namespace std;

class Book
{
private:
    string title;

public:
    Book(const string& title)
        : title(title)
    {
        cout << "Constructor: " << title << endl;
    }

    Book(const Book& other)
        : title(other.title)
    {
        cout << "Copy: " << title << endl;
    }

    ~Book()
    {
        cout << "Destructor: " << title << endl;
    }

    void setTitle(const string& newTitle)
    {
        title = newTitle;
    }

    void print() const
    {
        cout << title << endl;
    }
};

void read(Book book) { book.print(); }
void display(const Book& book) { book.print(); }
void update(Book& book) { book.setTitle("Updated"); }
```

נתונה הפונקציה הראשית:

```
int main()
{
    Book first("C++");

    read(first);
    display(first);

    Book second = first;
    update(second);

    first.print();
    second.print();

    return 0;
}
```

1.1 (8 נק) כתבו את הפלט המדויק של התוכנית, לפי סדר השורות. ציינו כל מקום בתוכנית שבו מופעל בנאי ההעתקה והסבירו מדוע.

1.2 (4 נק) השוו בין שתי הפונקציות הבאות. הסבירו כיצד מועבר האובייקט בכל אחת מהן, ומדוע הפונקציה השנייה מתאימה יותר כאשר נדרשת קריאה בלבד.

```
void read(Book book);
void display(const Book& book);
```

1.3 (3 נק) הסבירו את משמעות מילת המפתח המופיעה בסוף ההצהרה הבאה ומה היא מונעת מן הפונקציה לעשות.

```
void print() const;
```

2. נתונה המחלקה הבאה:

```
class IntBuffer
{
private:
    int* data;
    int size;

public:
    IntBuffer(int size);
    ~IntBuffer();
};
```

מימוש הבנאי והמפרק נתון:

```
IntBuffer::IntBuffer(int size)
    : size(size)
{
    data = new int[size];

    for (int i = 0; i < size; i++)
        data[i] = 0;
}

IntBuffer::~~IntBuffer()
{
    delete[] data;
}
```

2.1 (7 נק) נתון קטע הקוד הבא:

```
IntBuffer a(5);
IntBuffer b = a;

IntBuffer c(3);
c = a;
```

זהו את הבעיה בקטע הקוד והסבירו מה מתרחש בכל אחת משתי פעולות ההעתקה. התייחסו למצביעים הנשמרים באובייקטים, לזיכרון שהוקצה עבור c ולמה שעלול להתרחש בעת פירוק האובייקטים. הסבירו היטב!

2.2 (13 נק) תקנו את הבעיה במחלקה על ידי הוספת קוד מתאים ללא שינוי של פונקציות ומידע שכבר קיים. כתבו את ההצהרות ואת המימושים המלאים של הפונקציות שיש להוסיף.

3. נתונה המחלקה הבאה. ניתן להניח שכל ערכי הקלט אינם שליליים:

```
class Duration
{
private:
    int hours;
    int minutes;
    int seconds;

    void normalize()
    {
        int totalSeconds = hours * 3600 + minutes * 60 + seconds;

        hours = totalSeconds / 3600;
        minutes = (totalSeconds % 3600) / 60;
        seconds = totalSeconds % 60;
    }

public:
    Duration(int hours = 0, int minutes = 0, int seconds = 0)
        : hours(hours), minutes(minutes), seconds(seconds)
    {
        normalize();
    }

    int getHours() const
    {
        return hours;
    }

    int getMinutes() const
    {
        return minutes;
    }

    int getSeconds() const
    {
        return seconds;
    }
};
```

נרצה שהקוד הבא יתקמפל ויפעל ללא שינוי:

```
Duration d1(1, 50, 50);
Duration d2(0, 20, 30);

Duration d3 = d1 + d2;
d1 += d2;

if (d1 == d3)
    cout << "Equal" << endl;

cout << d3 << endl;
```

```
Equal
2 hours, 11 minutes, 20 seconds
```

3.1 (8 נק) זהו את כל האופרטורים החסרים שיש להוסיף למחלקה כדי שהקוד יתקמפל ויפיק את הפלט הנתון. כתבו את ההצהרה המתאימה לכל אופרטור וציינו האם הוא ימומש כמתודה של המחלקה או כפונקציה גלובלית.

3.2 (12 נק) ממשו את כל האופרטורים שזיהיתם בסעיף הקודם. ניתן להוסיף למחלקה את ההצהרות הדרושות, אך אין לשנות את את הקוד הנתון או הפלט הנדרש.

4. נתונות המחלקות הבאות:

```
class MediaItem
{
protected:
    string title;

public:
    MediaItem(const string& title) : title(title) {}
    virtual void play() const = 0;

    ~MediaItem()
    {
        cout << "MediaItem destroyed: " << title << endl;
    }
};
```

```
class Song : public MediaItem
{
public:
    Song(const string& title) : MediaItem(title) {}

    void play() const override
    {
        cout << "Playing song: " << title << endl;
    }

    ~Song()
    {
        cout << "Song destroyed: " << title << endl;
    }
};
```

```

class Podcast : public MediaItem
{
public:
    Podcast(const string& title) : MediaItem(title) {}

    void play() const override
    {
        cout << "Playing podcast: " << title << endl;
    }

    ~Podcast()
    {
        cout << "Podcast destroyed: " << title << endl;
    }
};

```

ובפונקציה הראשית:

```

int main()
{
    MediaItem* library[2];
    library[0] = new Song("Imagine");
    library[1] = new Podcast("C++ Today");

    for (int i = 0; i < 2; i++)
        library[i]->play();

    for (int i = 0; i < 2; i++)
        delete library[i];

    return 0;
}

```

4.1 (8 נק) הסבירו מהי מחלקה מופשטת והאם ניתן ליצור ממנה אובייקט באופן ישיר. כתבו את הפלט המדויק של לולאת ההפעלה (הראשונה) בלבד והסבירו מדוע מופעל מימוש שונה בכל איטרציה.

4.2 (8 נק) זהו את הבעיה המתרחשת בעת מחיקת האובייקטים דרך המצביעים למחלקת הבסיס והסבירו את הסיכון. תקנו את הבעיה באמצעות שינוי מתאים במחלקת הבסיס, ולאחר התיקון כתבו את הפלט המדויק שמתקבל מלולאת המחיקה.

4.3 (4 נק) לצורך סעיף זה בלבד הניחו שפונקציית `play` במחלקת הבסיס אינה מופשטת, יש לה מימוש והיא **אינה** וירטואלית. הסבירו מה יקרה להצהרות פונקציות `play` במחלקות יורשות כל עוד מילת המפתח **override** נשארת בהן. לאחר הסרתה, ציינו איזו פונקציה תופעל בקריאה דרך מצביע למחלקת הבסיס והסבירו מדוע.

5. ענו על הסעיפים הבאים:

5.1 (7 נק) כתבו פונקציה גנרית תבנית המקבלת מערך, גודל וערך לחיפוש, ומחזירה את מספר האיברים במערך השווים לערך הנתון. שם הפונקציה מופיע בדוגמאות השימוש. הפונקציה צריכה לתמוך, בין השאר, בשימושים הבאים:

```
int numbers[] = {1, 2, 1, 4, 1};  
cout << countMatches(numbers, 5, 1);           // 3  
  
string words[] = {"C++", "Java", "C++"};  
cout << countMatches(words, 3, string("C++")); // 2
```

5.2 (4 נק) נתונה רשימת שירים ולמחלקת השיר קיימת פונקציית הדפסה קבועה, כמוצג להלן. כתבו לולאה באמצעות איטרטור מפורש, ולא באמצעות לולאה מבוססת טווח, המדפיסה את כל השירים.

```
vector<Song> songs;  
void Song::print() const;
```

5.3 (4 נק) הסבירו מדוע אותה תבנית יכולה לעבוד הן עבור מספרים שלמים והן עבור מחרוזות. באיזו פעולה חייב לתמוך טיפוס האיברים כדי שהמימוש יתקמפל?

6. נתונה המחלקה הבאה:

```
class Playlist  
{  
private:  
    vector<Song> songs;  
  
public:  
    const Song& getSong(int index) const;  
};
```

6.1 (2 נק) ממשו את פונקציית הגישה כך שאם המיקום המבוקש אינו חוקי תיזרק החריגה התקנית המתאימה. אחרת תוחזר הפניה לשיר המתאים.

```
const Song& Playlist::getSong(int index) const;  
// Throw out_of_range when index is invalid.
```

6.2 (3 נק) כתבו קטע קוד הקורא לפונקציית הגישה ומטפל בחריגה באמצעות מבנה הטיפול המתאים.

6.3 (5 נק) לכל אחד מהמקרים הבאים בחרו במיכל המתאים ביותר מבין האפשרויות שלהלן, ונמקו בקצרה. מיכל אחר שנלמד יתקבל אם הבחירה מנומקת היטב.

vector	list	set	map
--------	------	-----	-----

בחרו מיכל עבור רשימת השמעה השומרת על סדר ההוספה, שעוברים עליה ברצף ולעיתים קרובות ניגשים לשיר לפי מיקומו, ונמקו את הבחירה

בחרו מיכל עבור קטלוג שירים שבו לכל שיר מספר קטלוגי ייחודי והפעולה המרכזית היא חיפוש שיר לפי מספר זה, ונמקו את הבחירה

ציינו יתרון אחד של שימוש במיכל מתאים מן הספרייה התקנית לעומת ניהול ידני של מערך דינמי כאשר מספר האיברים משתנה.

בהצלחה!